

Real-Time ADAS Perception and Risk Warning System

Project Plan, Setup Guide, Roadmap, and First Implementation Steps

Prepared for: Amanou Nasri

Target: Computer Vision / ADAS / Autonomous Vehicle Engineering thesis preparation

Version 1.0 - 2026-06-06

Important note: The uploaded attachment available for this planning session was the Autonomous Vehicle Engineering module handbook, not a conventional CV file. The plan is therefore aligned with your AVE curriculum, your stated thesis direction, and your known technical background: Python, PyTorch, OpenCV, Git/GitHub, Windows 11, RTX 3050, Google Colab, and automotive/ADAS interest.

Core strategy

Build a working engineering system first. Train and optimize later. No random notebook project. No fake "real-time" claims without FPS measurements.

Table of Contents

- 1. Project decision and strategic positioning
- 2. Final scope and deliverables
- 3. System architecture
- 4. Hardware, software, and workflow assumptions
- 5. Four-week first-demo plan
- 6. Twelve-week CV-grade plan
- 7. Phase 0 - Windows 11 setup, repository, and environment
- 8. Phase 1 - first object-detection pipeline
- 9. Phase 2 - object tracking pipeline
- 10. Phase 3 - ADAS warning engine
- 11. Phase 4 - demo polish, metrics, and logs
- 12. KITTI data strategy
- 13. Colab training plan for later phases
- 14. Evaluation plan
- 15. GitHub, README, reporting, slides, and CV outputs
- 16. Troubleshooting guide
- 17. Definition of done
- 18. Immediate next actions
- 19. References

1. Project decision and strategic positioning

Final project title: Real-Time ADAS Perception and Risk Warning System using Deep Learning

Later extension title: Real-Time ADAS Perception, Distance Estimation, and Lane-Aware Risk Assessment

Strategic reason: this project sits exactly at the intersection of computer vision, autonomous-driving perception, sensor data processing, ADAS logic, software engineering, and thesis preparation. It should be presented as an engineering pipeline, not merely as a deep-learning demo.

Decision	Chosen option	Reason
Goal	CV project that can later become thesis foundation	Strong enough for portfolio, controlled enough to finish.
Timeline	30 hours/week; first demo in 4 weeks	Enough time for clean repo, runnable scripts, video, logs, and metrics.
Direction	B then C then D	Start with ADAS risk warning, then add distance estimation, then lane awareness.
Dataset	KITTI first	Autonomous-driving benchmark with manageable size and clear relevance.
Training	Use pretrained model first	Training before pipeline is premature. Build the product skeleton first.
Setup	Windows coding + Colab training	Local RTX 3050 is enough for inference/debugging; Colab handles heavier training.
Evaluation level	Strong	The project must include FPS, logs, failure cases, and later mAP/tracking comparisons.

Blunt rule: the CV value comes from measured engineering output. A nice video alone is weak. A clean repo, reproducible commands, metrics, logs, and a technical report make it serious.

2. Final scope and deliverables

2.1 Version 1 scope - first public demo

Feature	Included in V1?	Implementation approach
Object detection	Yes	Ultralytics YOLO pretrained model.
Object tracking	Yes	ByteTrack first; BoT-SORT comparison later.
Front-zone detection	Yes	Polygon/rectangle risk zone in image coordinates.
Pedestrian/cyclist risk zone	Yes	Class-filtered intersection with central zone.
Vehicle-too-close warning	Yes	Approximate monocular heuristic from bbox size and bottom position.
Time-to-collision	Later	Requires distance history and stable tracked IDs.
Lane-based warning	Later	Requires lane/road-zone estimation.
YOLO fine-tuning	Later	After pipeline and logging are stable.
CARLA/ROS 2	Later	Extension only; not allowed to block V1.

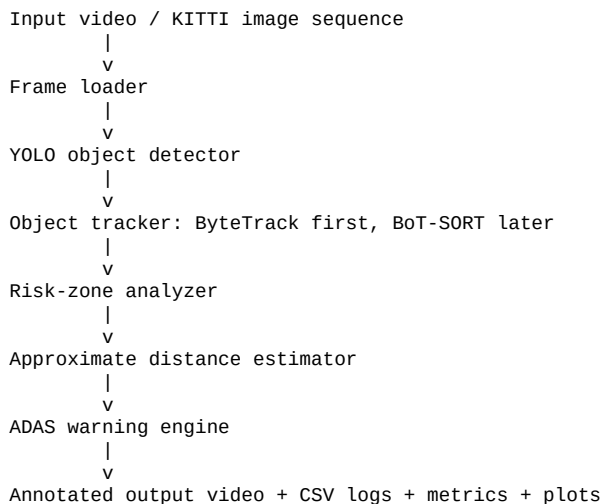
2.2 Final deliverables

Deliverable	Minimum standard
GitHub repository	Runnable from scripts, clear README, clean folder structure, no dataset dump.
Demo video	60-90 seconds, annotated detections/tracks/warnings/FPS.
Technical report PDF	Architecture, methodology, experiments, metrics, limitations, next work.
CV bullet points	2-3 quantified bullets with tools, metrics, and ADAS relevance.
LinkedIn post	Short technical announcement with demo video/GitHub.

Deliverable	Minimum standard
Presentation slides	6-8 slides for professor/recruiter explanation.
Thesis proposal idea	1-2 thesis directions derived from project extensions.

3. System architecture

The system must be built as a pipeline, not as disconnected scripts.



3.1 Warning hierarchy in V1

Priority	Warning	Trigger
High	PEDESTRIAN RISK	Pedestrian bbox center or bottom point enters pedestrian/cyclist risk zone.
High	CYCLIST RISK	Cyclist bbox center or bottom point enters risk zone.
Medium	VEHICLE TOO CLOSE	Vehicle bbox appears large/low enough to suggest short distance.
Low	FRONT OBJECT	Any vehicle/pedestrian/cyclist is inside front zone.

The initial distance estimate is deliberately approximate. You will label it as image-based heuristic distance estimation until KITTI calibration, stereo, or lidar information is added.

4. Hardware, software, and workflow assumptions

Item	Your setup	Project usage
OS	Windows 11	Main development environment.
GPU	RTX 3050	Local inference/debugging/light experiments.
Cloud	Google Colab free; Pro if needed	Fine-tuning and longer experiments.
Storage	External HDD + Google Drive 200 GB	Datasets and heavy outputs stored outside GitHub.
IDE	VS Code	Coding and repo management.
Environment	venv	Simple and clean Windows setup.
Core libraries	PyTorch, OpenCV, Ultralytics, NumPy, Pandas	Detection/tracking/video/metrics.

4.1 Tooling principle

- Use Windows locally for code, debugging, inference, visual checks, and GitHub work.
- Use Google Colab for training and heavier experiments only after V1 is stable.
- Use the external HDD for raw datasets and output videos. Do not commit heavy files to GitHub.

- Use Google Drive as Colab storage for KITTI training runs and model checkpoints.

5. Four-week first-demo plan

Week	Main objective	Concrete outputs
Week 1	Environment, repo, first YOLO inference	run_detection.py, annotated image/video output.
Week 2	Tracking and logs	run_tracking.py, tracked video, tracking_results.csv.
Week 3	ADAS warning logic	run_adas_demo.py, risk zones, warnings.csv.
Week 4	Demo polish and metrics	adas_demo_v1.mp4, README v1, FPS plot, metrics CSV.

5.1 Weekly working rhythm

Day type	Work allocation
Coding blocks	4-5 focused blocks per week, 2-3 hours each.
Testing blocks	Every feature must be tested on one image, one short video, and one driving sequence.
Documentation blocks	At least 2 hours/week updating README, notes, assumptions, and results.
Cleanup blocks	One repo cleanup session/week: remove dead scripts, rename confusing files, commit cleanly.

6. Twelve-week CV-grade plan

Phase	Weeks	Output
V1 - Demo pipeline	1-4	Detection + tracking + basic ADAS warning system.
V2 - Evaluation	5-6	FPS, confidence distribution, failure cases, warning statistics.
V3 - KITTI fine-tuning	7-8	YOLO fine-tuned on KITTI in Colab.
V4 - Distance estimation	9	Improved approximate distance using KITTI calibration or bbox heuristics.
V5 - Time-to-collision	10	TTC estimate from tracked object distance history.
V6 - Lane/risk-zone awareness	11	Improved risk zones using lane or road-area approximation.
V7 - Publication package	12	Technical report PDF, slides, CV bullets, LinkedIn post, thesis proposal.

7. Phase 0 - Windows 11 setup, repository, and environment

Goal: create a clean local project that can run object detection before any training is attempted.

7.1 Create project folder

```
mkdir C:\Users\amanu\Documents\Projects
cd C:\Users\amanu\Documents\Projects
mkdir adas-perception-risk-system
cd adas-perception-risk-system
```

7.2 Create GitHub repository

1. Go to GitHub.
2. Create a new repository named adas-perception-risk-system.
3. Set visibility to Public only when the first clean demo is ready. Private is acceptable during development.
4. Do not initialize with a README if you want to push an existing local repo manually, or initialize and clone if you prefer the GitHub-first workflow.

```
git init
git branch -M main
git status
```

7.3 Create virtual environment

```
py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1
```

```
python --version
python -m pip install --upgrade pip setuptools wheel
```

If py -3.11 is not available, install Python 3.11 or 3.10. Do not use a random Python version because PyTorch support depends on Python and CUDA compatibility.

7.4 Install PyTorch with CUDA support

Use the official PyTorch selector for the latest exact command. The selector is the source of truth because Python, CUDA, and wheel versions change. Current PyTorch documentation states that Windows pip installation is supported and that recent stable versions require Python 3.10 or later.

```
# Open the PyTorch Get Started page and select:
# OS: Windows
# Package: Pip
# Language: Python
# Compute Platform: CUDA version suitable for your machine
# Then run the generated command.

# Example pattern only. Do not blindly trust it forever:
pip3 install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu128
```

7.5 Verify GPU access

```
python -c "import torch; print('Torch:', torch.__version__); print('CUDA:', torch.cuda.is_available());
print(torch.cuda.get_device_name(0) if torch.cuda.is_available() else 'No GPU')"
```

If CUDA available is False, do not panic. The project can still start with CPU inference, but GPU support should be fixed before larger local experiments.

7.6 Install project libraries

```
pip install ultralytics opencv-python numpy pandas matplotlib pyyaml tqdm rich
pip freeze > requirements.txt
```

7.7 Create .gitignore

```
# PowerShell
@"
.venv/
__pycache__/
*.pyc
.ipynb_checkpoints/

# datasets and heavy outputs
data/raw/
data/kitti/
outputs/demo/*.mp4
outputs/detections/*.mp4
outputs/tracking/*.mp4
outputs/runs/
runs/

# model weights
*.pt
*.onnx
*.engine

# OS/editor
.DS_Store
.vscode/
"@ | Out-File -Encoding utf8 .gitignore
```

7.8 Create repository structure

```
mkdir configs, data, data\samples, notebooks, scripts, src, outputs, reports, slides
mkdir outputs\demo, outputs\detections, outputs\tracking, outputs\logs, outputs\figures, outputs\reports
mkdir src\adas_perception
mkdir src\adas_perception\detection, src\adas_perception\tracking
mkdir src\adas_perception\risk, src\adas_perception\visualization, src\adas_perception\utils
ni src\adas_perception\__init__.py
ni src\adas_perception\detection\__init__.py
ni src\adas_perception\tracking\__init__.py
ni src\adas_perception\risk\__init__.py
ni src\adas_perception\visualization\__init__.py
ni src\adas_perception\utils\__init__.py
```

7.9 First commit

```
git add .
git commit -m "Initialize ADAS perception project structure"
```

8. Phase 1 - first object-detection pipeline

Goal: run YOLO detection on one image and one video, then save annotated outputs automatically.

8.1 Create detector config

```
# configs/detector.yaml
model: yolo11s.pt          # use yolo11n.pt if laptop is slow
imgsz: 640
conf: 0.35
iou: 0.50
classes_of_interest:
  - car
  - truck
  - bus
  - motorcycle
  - bicycle
  - person
```

8.2 Create scripts/run_detection.py

```
from pathlib import Path
import argparse
from ultralytics import YOLO

def parse_args():
    parser = argparse.ArgumentParser()
    parser.add_argument("--source", required=True, help="Image, video, webcam index, or folder")
    parser.add_argument("--model", default="yolo11s.pt")
    parser.add_argument("--conf", type=float, default=0.35)
    parser.add_argument("--imgsz", type=int, default=640)
    parser.add_argument("--out", default="outputs/detections")
    return parser.parse_args()

def main():
    args = parse_args()
    Path(args.out).mkdir(parents=True, exist_ok=True)
    model = YOLO(args.model)
    results = model.predict(
        source=args.source,
        conf=args.conf,
        imgsz=args.imgsz,
        save=True,
        project=args.out,
        name="predict",
        exist_ok=True,
    )
    print(f"Finished detection. Results saved under: {args.out}")
    print(f"Number of result objects returned: {len(results)}")

if __name__ == "__main__":
    main()
```

8.3 Test with webcam or sample video

```
# Put a short video into data/samples/test_video.mp4 first.
python scripts/run_detection.py --source data/samples/test_video.mp4

# Optional webcam test:
python scripts/run_detection.py --source 0
```

8.4 Phase 1 acceptance criteria

- The script runs from PowerShell without path hacks.
- Annotated results are saved under outputs/detections/predict.
- The README contains the exact command that worked.
- The first result is committed without model weights or videos larger than GitHub should handle.

9. Phase 2 - object tracking pipeline

Goal: object IDs stay consistent across frames and tracking output is saved as video plus CSV logs.

9.1 Create tracker config

```
# configs/tracker.yaml
tracker: bytetrack.yaml
model: yolo11s.pt
conf: 0.35
imgsz: 640
```

```
persist: true
```

9.2 Create scripts/run_tracking.py

```
from pathlib import Path
import argparse
import csv
from ultralytics import YOLO

def parse_args():
    parser = argparse.ArgumentParser()
    parser.add_argument("--source", required=True)
    parser.add_argument("--model", default="yolo11s.pt")
    parser.add_argument("--tracker", default="bytetrack.yaml")
    parser.add_argument("--conf", type=float, default=0.35)
    parser.add_argument("--imgsz", type=int, default=640)
    parser.add_argument("--out", default="outputs/tracking")
    return parser.parse_args()

def main():
    args = parse_args()
    Path(args.out).mkdir(parents=True, exist_ok=True)
    Path("outputs/logs").mkdir(parents=True, exist_ok=True)
    model = YOLO(args.model)
    results = model.track(
        source=args.source,
        tracker=args.tracker,
        conf=args.conf,
        imgsz=args.imgsz,
        save=True,
        project=args.out,
        name="track",
        exist_ok=True,
        stream=True,
        persist=True,
    )
    csv_path = Path("outputs/logs/tracking_results.csv")
    with csv_path.open("w", newline="", encoding="utf-8") as f:
        writer = csv.writer(f)
        writer.writerow(["frame", "track_id", "class_id", "class_name", "confidence", "x1", "y1", "x2",
            "y2"])
        for frame_idx, result in enumerate(results):
            names = result.names
            if result.bboxes is None:
                continue
            for box in result.bboxes:
                if box.id is None:
                    continue
                track_id = int(box.id.item())
                cls_id = int(box.cls.item())
                conf = float(box.conf.item())
                x1, y1, x2, y2 = [float(v) for v in box.xyxy[0].tolist()]
                writer.writerow([frame_idx, track_id, cls_id, names[cls_id], conf, x1, y1, x2, y2])
    print(f"Tracking finished. CSV saved to {csv_path}")

if __name__ == "__main__":
    main()
```

9.3 Run tracking

```
python scripts/run_tracking.py --source data/samples/test_video.mp4 --tracker bytetrack.yaml
python scripts/run_tracking.py --source data/samples/test_video.mp4 --tracker botsort.yaml
```

9.4 Phase 2 acceptance criteria

- Tracked video is saved.
- tracking_results.csv contains frame numbers, track IDs, classes, confidences, and bbox coordinates.
- At least one tracked object keeps the same ID for multiple frames.
- You compare ByteTrack and BoT-SORT qualitatively in notes.

10. Phase 3 - ADAS warning engine

Goal: turn detection/tracking into an ADAS-like perception prototype with explicit warning logic.

10.1 Define risk zones

```
# configs/risk_zones.yaml
image_width: 1280
image_height: 720

front_zone:
```



```

type: polygon
points:
  - [480, 720]
  - [800, 720]
  - [740, 420]
  - [540, 420]

pedestrian_cyclist_zone:
  type: polygon
  points:
    - [360, 720]
    - [920, 720]
    - [820, 360]
    - [460, 360]

close_vehicle:
  min_bbox_height_ratio: 0.25
  min_bottom_y_ratio: 0.65

```

10.2 Warning logic

Function	Input	Output
is_in_front_zone	bbox center/bottom point + front polygon	True/False
is_pedestrian_or_cyclist_risk	class name + bbox + risk polygon	True/False
is_vehicle_too_close	class name + bbox height + bottom y	True/False
assign_warning_priority	all triggered warnings	highest-priority warning label
write_warning_log	frame, track_id, class, warning, bbox	CSV row

10.3 Create src/adas_perception/risk/warning_engine.py

```

def bbox_bottom_center(box):
    x1, y1, x2, y2 = box
    return ((x1 + x2) / 2, y2)

def point_in_polygon(point, polygon):
    x, y = point
    inside = False
    j = len(polygon) - 1
    for i in range(len(polygon)):
        xi, yi = polygon[i]
        xj, yj = polygon[j]
        intersects = ((yi > y) != (yj > y)) and (x < (xj - xi) * (y - yi) / ((yj - yi) + 1e-9) + xi)
        if intersects:
            inside = not inside
        j = i
    return inside

def vehicle_too_close(class_name, box, image_h, min_height_ratio=0.25, min_bottom_ratio=0.65):
    vehicle_classes = {"car", "truck", "bus", "motorcycle", "van"}
    if class_name not in vehicle_classes:
        return False
    x1, y1, x2, y2 = box
    height_ratio = (y2 - y1) / image_h
    bottom_ratio = y2 / image_h
    return height_ratio >= min_height_ratio and bottom_ratio >= min_bottom_ratio

def get_warnings(class_name, box, image_h, front_polygon, ped_cyc_polygon):
    warnings = []
    bottom = bbox_bottom_center(box)
    if point_in_polygon(bottom, front_polygon):
        warnings.append("FRONT OBJECT")
    if class_name in {"person", "pedestrian", "cyclist", "bicycle"}:
        if point_in_polygon(bottom, ped_cyc_polygon):
            warnings.append("CYCLIST RISK" if class_name in {"cyclist", "bicycle"} else "PEDESTRIAN RISK")
    if vehicle_too_close(class_name, box, image_h):
        warnings.append("VEHICLE TOO CLOSE")
    return warnings

```

10.4 Phase 3 acceptance criteria

- Annotated video shows front zone and pedestrian/cyclist risk zone.
- Warnings are printed visually on frames.
- warnings.csv logs frame, object ID, class, warning type, and bbox.
- False warnings are expected at this stage; they must be documented, not hidden.

11. Phase 4 - demo polish, metrics, and logs

Goal: turn the first prototype into something presentable.

11.1 Required output files

outputs/demo/adas_demo_v1.mp4
outputs/logs/tracking_results.csv
outputs/logs/warnings.csv
outputs/reports/demo_metrics.csv
outputs/figures/fps_plot.png
README.md

11.2 Metrics to calculate in V1

Metric	Meaning
Average FPS	Whether the system can honestly be called near real-time.
Average inference time	Speed per frame, useful for hardware comparison.
Class counts	How many cars/pedestrians/cyclists were detected.
Track count	How many unique object IDs appeared.
Warning count	How many warnings of each type were triggered.
Warning duration	How many frames each risky object remained risky.

11.3 Demo quality checklist

- Use a short, visually clear driving clip.
- Overlay FPS, active warnings, object IDs, and zone polygons.
- Avoid clutter: do not show every tiny confidence value if it makes the video unreadable.
- Show at least one successful front-zone warning.
- Show at least one pedestrian/cyclist risk example if the video contains such objects.
- Export the video at reasonable quality, not blurry compression trash.

12. KITTI data strategy

Use KITTI in controlled layers. Do not download everything blindly.

Layer	Purpose	Where stored
Tiny sample	Debug code quickly	data/samples local repo or small ignored folder.
KITTI detection subset	Fine-tuning and validation later	External HDD + Google Drive.
KITTI tracking sequences	Tracking and later TTC work	External HDD + Google Drive.
Demo videos	Portfolio output	External HDD, selected final demo in outputs.

12.1 KITTI usage policy

- Do not commit KITTI data to GitHub.
- Do not commit raw video dumps.
- Keep a data/README.md explaining how to obtain the data.
- Use symlinks or config paths if datasets live on external HDD.
- Use Google Drive only for Colab training data and run outputs.

12.2 data/README.md template

Data

This repository does not include full datasets.

Recommended data sources:

- Small local sample videos in data/samples/ for quick debugging.

- KITTI object detection data for later fine-tuning/evaluation.
- KITTI tracking sequences for tracking and TTC experiments.

Local expected structure:

```
data/
  samples/
    test_video.mp4
  kitti/                # ignored by Git
    images/
    labels/
```

Large datasets are stored externally and are not committed.

13. Colab training plan for later phases

Colab is not needed for V1. Use it only when the local pipeline is already working.

13.1 Colab notebook skeleton

```
from google.colab import drive
drive.mount('/content/drive')

!pip install ultralytics

from ultralytics import YOLO

model = YOLO('yolo11s.pt') # or newer supported Ultralytics model

results = model.train(
    data='kitti.yaml',
    epochs=50,
    imgsz=640,
    batch=16,
    project='/content/drive/MyDrive/adas_project/runs',
    name='kitti_yolo_finetune'
)
```

13.2 Colab rules

- Save all trained weights to Google Drive, not the temporary Colab VM.
- Record every experiment: model, dataset, image size, batch size, epochs, runtime, and result metrics.
- Do not start 100-epoch runs before a 5-epoch smoke test succeeds.
- Use Colab Pro only if free tier interruptions become a serious bottleneck.

14. Evaluation plan

14.1 V1/V2 evaluation

Category	Metric	How to collect
Runtime	FPS	Measure frame processing time in run_adas_demo.py.
Runtime	Average inference time	Time model inference separately if possible.
Detection	Class count	Count detections by class from results/logs.
Detection	Confidence distribution	Plot histogram of confidence values.
Tracking	Number of unique tracks	Count unique track_id in tracking CSV.
Tracking	Average track length	Group by track_id and count frames.
Warning	Warning counts	Count warnings by type in warnings.csv.
Warning	Risk duration	Group warnings by track_id and count continuous frames.
Robustness	Failure cases	Save screenshots where the system is wrong.

14.2 Later thesis-grade evaluation

Area	Metric/experiment
Detection	mAP, precision, recall after KITTI fine-tuning.

Area	Metric/experiment
Model comparison	YOLO nano/small/medium comparison: accuracy vs FPS.
Tracker comparison	ByteTrack vs BoT-SORT: qualitative ID stability and later quantitative metrics.
Distance estimation	Heuristic vs calibration-based approximate distance, if KITTI calibration is used.
TTC	False positives and false negatives under changing traffic scenarios.
Deployment	ONNX export and inference-time comparison.

15. GitHub, README, reporting, slides, and CV outputs

15.1 Git workflow

```
git status
git add .
git commit -m "Add YOLO detection pipeline"
git commit -m "Add object tracking and CSV logging"
git commit -m "Add ADAS warning engine"
git commit -m "Add demo metrics and README documentation"
```

Commit messages should describe engineering milestones, not emotional progress. Bad: "finally works". Good: "Add ADAS warning engine".

15.2 README structure

```
# Real-Time ADAS Perception and Risk Warning System

## Overview
## Demo
## Features
## Architecture
## Installation
## Usage
## Dataset
## Results
## Limitations
## Roadmap
## References
```

15.3 Technical report structure

Section	Content
Abstract	Short summary of goal, methods, and results.
Introduction	ADAS perception problem and motivation.
System architecture	Detector, tracker, warning engine, logging.
Implementation	Libraries, repo structure, configs.
Experiments	Data, runtime, tracking, warning tests.
Results	FPS, examples, plots, warning statistics.
Failure cases	Where it fails and why.
Limitations	Monocular distance approximation, no lane model yet.
Future work	KITTI fine-tuning, TTC, lane-aware risk, ROS/CARLA.

15.4 Slide deck structure

Slide	Title
1	Problem and motivation
2	System architecture
3	Detection and tracking pipeline
4	ADAS warning logic
5	Results and demo screenshots

Slide	Title
6	Evaluation metrics and failure cases
7	Roadmap to thesis extension
8	Conclusion

15.5 Target CV bullets

- Developed a real-time ADAS perception and risk-warning pipeline using YOLO-based object detection and multi-object tracking on autonomous-driving video data.
- Implemented front-zone, pedestrian/cyclist, and close-vehicle warning logic with annotated video output, frame-level CSV logs, and FPS/runtime evaluation.
- Prepared the system for KITTI fine-tuning, distance estimation, time-to-collision analysis, and lane-aware risk assessment as a bachelor-thesis foundation.

16. Troubleshooting guide

Problem	Likely cause	Fix
torch.cuda.is_available() is False	Wrong PyTorch wheel or driver mismatch	Reinstall from official PyTorch selector; update NVIDIA driver if needed.
Ultralytics model does not download	Network issue or unsupported model name	Try yolo11n.pt/yolo11s.pt; update ultralytics.
Video output not created	Source path wrong or codec issue	Use absolute path; test with a small MP4; check outputs folder.
Tracking IDs disappear quickly	Low confidence, occlusion, bad video quality	Lower conf slightly; compare ByteTrack and BoT-SORT.
Warnings trigger too often	Risk zone too wide or thresholds too loose	Narrow polygons and increase close-vehicle thresholds.
Laptop is slow	Model too large or high resolution	Use yolo11n.pt and imgsz 480/640; process shorter clips.
GitHub repo too large	Videos/datasets/weights committed	Remove from Git history if necessary; use .gitignore and external storage.

17. Definition of done

17.1 V1 is done only when all are true

- Detection script runs from command line.
- Tracking script runs from command line.
- ADAS demo script runs from command line.
- Annotated demo video exists.
- tracking_results.csv exists.
- warnings.csv exists.
- Average FPS is reported.
- README explains installation and usage.
- At least 3 failure cases are documented.
- GitHub repo is clean and does not contain datasets or heavy weights.

17.2 CV-grade project is done only when all are true

- V1 is complete.
- Metrics are summarized in tables/plots.
- Technical report PDF is written.

- Demo video is short, clear, and public-ready.
- README has architecture diagram and result screenshots.
- CV bullets include methods and measured outputs.
- At least one extension path is clearly thesis-worthy: KITTI fine-tuning, TTC, lane-aware warning, or CARLA/ROS 2 integration.

18. Immediate next actions

1. Create the local project folder.
2. Initialize Git.
3. Create the GitHub repo.
4. Create and activate .venv.
5. Install PyTorch using the official selector.
6. Verify torch.cuda.is_available().
7. Install Ultralytics and OpenCV.
8. Create the folder structure.
9. Add .gitignore.
10. Create scripts/run_detection.py.
11. Place one short test video into data/samples/.
12. Run first detection command.
13. Commit the working baseline.

The first milestone command you must make work:
python scripts/run_detection.py --source data/samples/test_video.mp4

Once this command works and saves an annotated output, Phase 1 is real. Before that, the project is only an idea.

19. References

- PyTorch. Get Started Locally. <https://pytorch.org/get-started/locally/>
- Ultralytics. Multi-Object Tracking with Ultralytics YOLO. <https://docs.ultralytics.com/modes/track>
- Ultralytics. KITTI Dataset for YOLO detection. <https://docs.ultralytics.com/datasets/detect/kitti/>
- KITTI Vision Benchmark Suite. <https://www.cvlibs.net/datasets/kitti/>
- KITTI Object Detection Evaluation. https://www.cvlibs.net/datasets/kitti/eval_3dobject.php
- KITTI Object Tracking Evaluation. https://www.cvlibs.net/datasets/kitti/eval_tracking.php